

q_mup_coding

December 10, 2025

```
[6]: !pip install torch torchvision torchaudio keras matplotlib numpy tensorflow
```

```
Requirement already satisfied: torch in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages
(2.9.1)
Requirement already satisfied: torchvision in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages
(0.24.1)
Requirement already satisfied: torchaudio in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages
(2.9.1)
Requirement already satisfied: keras in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages
(3.12.0)
Requirement already satisfied: matplotlib in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages
(3.10.0)
Requirement already satisfied: numpy in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages
(2.2.2)
Collecting tensorflow
  Using cached tensorflow-2.20.0-cp313-cp313-macosx_12_0_arm64.whl.metadata (4.5
kB)
Requirement already satisfied: filelock in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
torch) (3.20.0)
Requirement already satisfied: typing-extensions>=4.10.0 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
torch) (4.15.0)
Requirement already satisfied: setuptools in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
torch) (80.9.0)
Requirement already satisfied: sympy>=1.13.3 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
torch) (3.4.2)
```

Requirement already satisfied: jinja2 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
torch) (3.1.6)

Requirement already satisfied: fsspec>=0.8.5 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
torch) (2025.12.0)

Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
torchvision) (11.1.0)

Requirement already satisfied: absl-py in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
keras) (2.3.1)

Requirement already satisfied: rich in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
keras) (14.2.0)

Requirement already satisfied: namex in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
keras) (0.1.0)

Requirement already satisfied: h5py in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
keras) (3.15.1)

Requirement already satisfied: optree in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
keras) (0.18.0)

Requirement already satisfied: ml-dtypes in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
keras) (0.5.4)

Requirement already satisfied: packaging in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
keras) (24.2)

Requirement already satisfied: contourpy>=1.0.1 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
matplotlib) (1.3.1)

Requirement already satisfied: cycler>=0.10 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
matplotlib) (4.55.6)

Requirement already satisfied: kiwisolver>=1.3.1 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
matplotlib) (1.4.8)

Requirement already satisfied: pyparsing>=2.3.1 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
matplotlib) (3.2.1)

Requirement already satisfied: python-dateutil>=2.7 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
matplotlib) (2.9.0.post0)

Collecting astunparse>=1.6.0 (from tensorflow)
 Using cached astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
 Collecting flatbuffers>=24.3.25 (from tensorflow)
 Using cached flatbuffers-25.9.23-py2.py3-none-any.whl.metadata (875 bytes)
 Collecting gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 (from tensorflow)
 Downloading gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
 Collecting google_pasta>=0.1.1 (from tensorflow)
 Using cached google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
 Collecting libclang>=13.0.0 (from tensorflow)
 Using cached libclang-18.1.1-1-py2.py3-none-macosx_11_0_arm64.whl.metadata (5.2 kB)
 Collecting opt_einsum>=2.3.2 (from tensorflow)
 Using cached opt_einsum-3.4.0-py3-none-any.whl.metadata (6.3 kB)
 Collecting protobuf>=5.28.0 (from tensorflow)
 Downloading protobuf-6.33.2-cp39-abi3-macosx_10_9_universal2.whl.metadata (593 bytes)
 Requirement already satisfied: requests<3,>=2.21.0 in
 /Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
 tensorflow) (2.32.3)
 Requirement already satisfied: six>=1.12.0 in
 /Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
 tensorflow) (1.17.0)
 Collecting termcolor>=1.1.0 (from tensorflow)
 Downloading termcolor-3.2.0-py3-none-any.whl.metadata (6.4 kB)
 Collecting wrapt>=1.11.0 (from tensorflow)
 Downloading wrapt-2.0.1-cp313-cp313-macosx_11_0_arm64.whl.metadata (9.0 kB)
 Collecting grpcio<2.0,>=1.24.3 (from tensorflow)
 Downloading grpcio-1.76.0-cp313-cp313-macosx_11_0_universal2.whl.metadata (3.7
 kB)
 Collecting tensorboard~2.20.0 (from tensorflow)
 Using cached tensorboard-2.20.0-py3-none-any.whl.metadata (1.8 kB)
 Collecting wheel<1.0,>=0.23.0 (from astunparse>=1.6.0->tensorflow)
 Using cached wheel-0.45.1-py3-none-any.whl.metadata (2.3 kB)
 Requirement already satisfied: charset-normalizer<4,>=2 in
 /Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
 requests<3,>=2.21.0->tensorflow) (3.4.1)
 Requirement already satisfied: idna<4,>=2.5 in
 /Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
 requests<3,>=2.21.0->tensorflow) (3.10)
 Requirement already satisfied: urllib3<3,>=1.21.1 in
 /Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
 requests<3,>=2.21.0->tensorflow) (2.3.0)
 Requirement already satisfied: certifi>=2017.4.17 in
 /Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
 requests<3,>=2.21.0->tensorflow) (2024.12.14)
 Requirement already satisfied: mpmath<1.4,>=1.1.0 in
 /Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
 sympy>=1.13.3->torch) (1.3.0)

```

Collecting markdown>=2.6.8 (from tensorboard~=2.20.0->tensorflow)
  Downloading markdown-3.10-py3-none-any.whl.metadata (5.1 kB)
Collecting tensorboard-data-server<0.8.0,>=0.7.0 (from
tensorboard~=2.20.0->tensorflow)
  Using cached tensorboard_data_server-0.7.2-py3-none-any.whl.metadata (1.1 kB)
Collecting werkzeug>=1.0.1 (from tensorboard~=2.20.0->tensorflow)
  Downloading werkzeug-3.1.4-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: MarkupSafe>=2.0 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
jinja2->torch) (3.0.3)
Requirement already satisfied: markdown-it-py>=2.2.0 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
rich->keras) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
rich->keras) (2.19.1)
Requirement already satisfied: mdurl~=0.1 in
/Users/gg027/Desktop/SP25/CS189/hws/hw1/myenv/lib/python3.13/site-packages (from
markdown-it-py>=2.2.0->rich->keras) (0.1.2)
Using cached tensorflow-2.20.0-cp313-cp313-macosx_12_0_arm64.whl (200.7 MB)
Using cached astunparse-1.6.3-py2.py3-none-any.whl (12 kB)
Using cached flatbuffers-25.9.23-py2.py3-none-any.whl (30 kB)
Downloading gast-0.7.0-py3-none-any.whl (22 kB)
Using cached google_pasta-0.2.0-py3-none-any.whl (57 kB)
Downloading grpcio-1.76.0-cp313-cp313-macosx_11_0_universal2.whl (11.8 MB)
11.8/11.8 MB
10.4 MB/s eta 0:00:00:00:01m
Using cached libclang-18.1.1-1-py2.py3-none-macosx_11_0_arm64.whl (25.8
MB)
Using cached opt_einsum-3.4.0-py3-none-any.whl (71 kB)
Downloading protobuf-6.33.2-cp39-abi3-macosx_10_9_universal2.whl (427 kB)
Using cached tensorboard-2.20.0-py3-none-any.whl (5.5 MB)
Downloading termcolor-3.2.0-py3-none-any.whl (7.7 kB)
Downloading wrapt-2.0.1-cp313-cp313-macosx_11_0_arm64.whl (61 kB)
Downloading markdown-3.10-py3-none-any.whl (107 kB)
Using cached tensorboard_data_server-0.7.2-py3-none-any.whl (2.4 kB)
Downloading werkzeug-3.1.4-py3-none-any.whl (224 kB)
Using cached wheel-0.45.1-py3-none-any.whl (72 kB)
Installing collected packages: libclang, flatbuffers, wrapt, wheel, werkzeug,
termcolor, tensorboard-data-server, protobuf, opt_einsum, markdown, grpcio,
google_pasta, gast, tensorboard, astunparse, tensorflow
Successfully installed astunparse-1.6.3 flatbuffers-25.9.23 gast-0.7.0
google_pasta-0.2.0 grpcio-1.76.0 libclang-18.1.1 markdown-3.10 opt_einsum-3.4.0
protobuf-6.33.2 tensorboard-2.20.0 tensorboard-data-server-0.7.2
tensorflow-2.20.0 termcolor-3.2.0 werkzeug-3.1.4 wheel-0.45.1 wrapt-2.0.1

[notice] A new release of pip is
available: 25.0 -> 25.3

```

[notice] To update, run:
`pip install --upgrade pip`

0.1 Maximal Update Parameterization

In this problem, we will examine the training of a simple MLP with hidden layers of varying widths. We will then investigate the maximal update parameterization (muP) which will allow us to use a single global learning rate to jointly train layers of any width.

Note: This homework question is new this year and it is messier than usual. We felt it was worth it to get it out so you can play with these new techniques. If you're feeling stuck, don't hesitate to ask questions on Ed.

```
[7]: import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from keras.datasets import mnist
import matplotlib.pyplot as plt

%matplotlib inline

# Device configuration
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

class MLP(nn.Module):
    def __init__(self, input_size=784, hidden_sizes = [8, 16, 32, 64, 128],
        num_classes=10):
        super(MLP, self).__init__()
        all_hidden_sizes = [input_size] + hidden_sizes + [num_classes]
        self.layers = nn.ModuleList()
        for i in range(len(all_hidden_sizes)-1):
            self.layers.append(nn.Linear(all_hidden_sizes[i],
        all_hidden_sizes[i+1]))
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        activations = []
        x = x.view(x.size(0), -1) # Flatten: (batch_size, 28*28)
        for layer in self.layers[:-1]:
            x = self.sigmoid(layer(x))
            activations.append(x)
        x = self.layers[-1](x)
        activations = activations[1:]
        return x, [a.detach() for a in activations]

# Load MNIST data
(train_images, train_labels), (valid_images, valid_labels) = mnist.load_data()
```

```

# Normalize pixel values to [0, 1]
train_images = train_images.astype(np.float32) / 255.0
valid_images = valid_images.astype(np.float32) / 255.0

# Convert to PyTorch tensors
train_images = torch.from_numpy(train_images)
train_labels = torch.from_numpy(train_labels).long()
valid_images = torch.from_numpy(valid_images)
valid_labels = torch.from_numpy(valid_labels).long()

def rms(x, dim):
    return torch.sqrt(torch.mean(x**2, dim=dim))

```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>
11490434/11490434 1s
0us/step

```

[8]: from torch.optim.optimizer import Optimizer
from typing import Any
class SimpleAdam(Optimizer):
    def __init__(
        self,
        params: Any,
        lr: float = 1e-1,
        b1: float = 0.9,
        b2: float = 0.999,
    ):
        defaults = dict(lr=lr, b1=b1, b2=b2,)
        super(SimpleAdam, self).__init__(params, defaults)

    @torch.no_grad()
    def step(self):
        for group in self.param_groups:
            for p in group['params']:
                grad = p.grad.data

                state = self.state[p]
                if len(state) == 0: # Initialization
                    state["step"] = torch.tensor(0.0)
                    state['momentum'] = torch.zeros_like(p)
                    state['variance'] = torch.zeros_like(p)

                state['step'] += 1
                m = state['momentum']
                m.lerp_(grad, 1-group["b1"])

```

```

        v = state['variance']
        v.lerp_(grad**2, 1-group["b2"])

        m_hat = m / (1 - group["b1"]**state['step'])
        v_hat = v / (1 - group["b2"]**state['step'])
        u = m_hat / (torch.sqrt(v_hat) + 1e-16)

        p.add_(u, alpha=-group['lr'])
    return None

```

```

[9]: batch_idx = np.random.randint(0, len(train_images), size=64)
def train_one_step(mlp=MLP, hiddens=[8, 16, 64, 64, 64, 256, 256, 1024],
    ↪optimizer=SimpleAdam, label="Adam", lr=0.01):
    model = mlp(hidden_sizes=hiddens).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optimizer(model.parameters(), lr=lr)

    prev_activations = None
    for i in range(2):
        images_batch = train_images[batch_idx]
        labels_batch = train_labels[batch_idx]
        images_batch, labels_batch = images_batch.to(device), labels_batch.
    ↪to(device)

        optimizer.zero_grad()
        outputs, activations = model(images_batch)
        loss = criterion(outputs, labels_batch)
        loss.backward()
        optimizer.step()

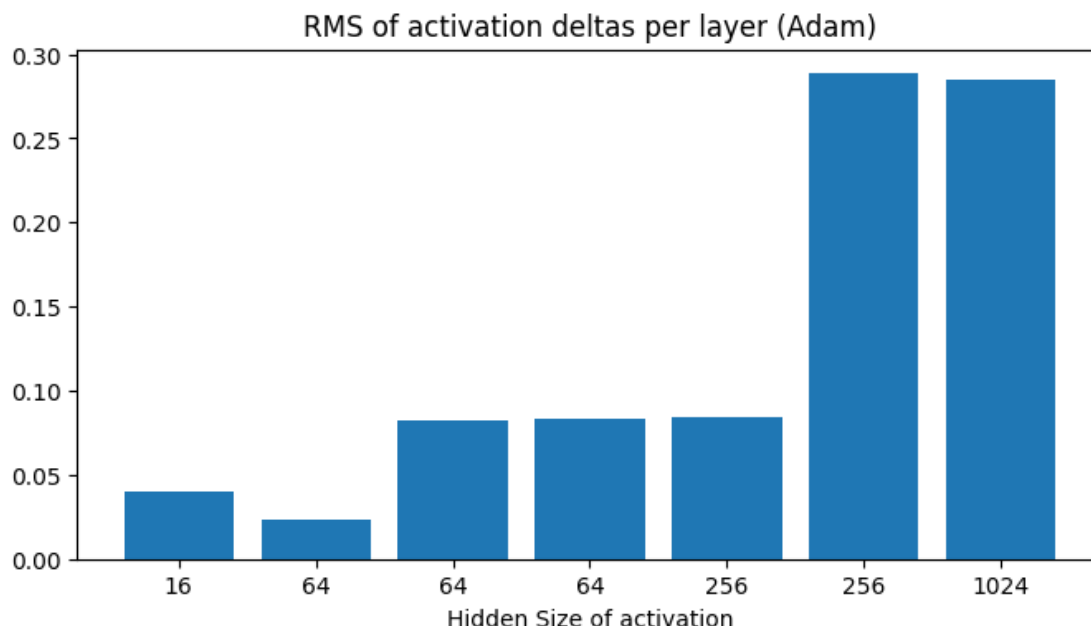
        if i > 0:
            print([a.shape for a in activations])
            activation_deltas = [a - pa for a, pa in zip(activations,
    ↪prev_activations)]
            activation_deltas_rms = [torch.mean(rms(a, dim=-1)) for a in
    ↪activation_deltas]
            prev_activations = activations

    # plot deltas
    deltas = np.array(activation_deltas_rms)
    fig, axs = plt.subplots(1, figsize=(8, 4))
    axs.set_title(f'RMS of activation deltas per layer ({label})')
    axs.set_xlabel('Hidden Size of activation')
    axs.bar(np.arange(deltas.shape[0]), deltas)
    axs.set_xticks(np.arange(deltas.shape[0]))
    axs.set_xticklabels(hiddens[1:])
    plt.show()

```

```
train_one_step(optimizer=SimpleAdam)
```

```
[torch.Size([64, 16]), torch.Size([64, 64]), torch.Size([64, 64]),  
torch.Size([64, 64]), torch.Size([64, 256]), torch.Size([64, 256]),  
torch.Size([64, 1024])]
```



0.2 a. Examining the norms of a heterogenous MLP.

Run the above cell, which trains a neural network for a single gradient step, then examines the effect of that step on the resulting activations. What are the dimensions of each layer in the neural network?

Answer: The network processes a 784-dimensional flattened image and then passes through hidden widths [8, 16, 64, 64, 64, 256, 256, 1024] before finishing with a 10-way classifier, so the linear layers are 784->8->16->64->64->64->256->256->1024->10.

How does the dimensionality of the layer affect the RMS norm of the activation deltas?

Answer: The bar chart (cell_4_1.png) shows that the early 16/64-width layers only move about 0.02–0.09 RMS, while the 256/1024 layers jump to roughly 0.29. In other words, once the layer width grows the RMS delta grows almost monotonically, so wider representations receive much larger updates when everything shares the same global learning rate.

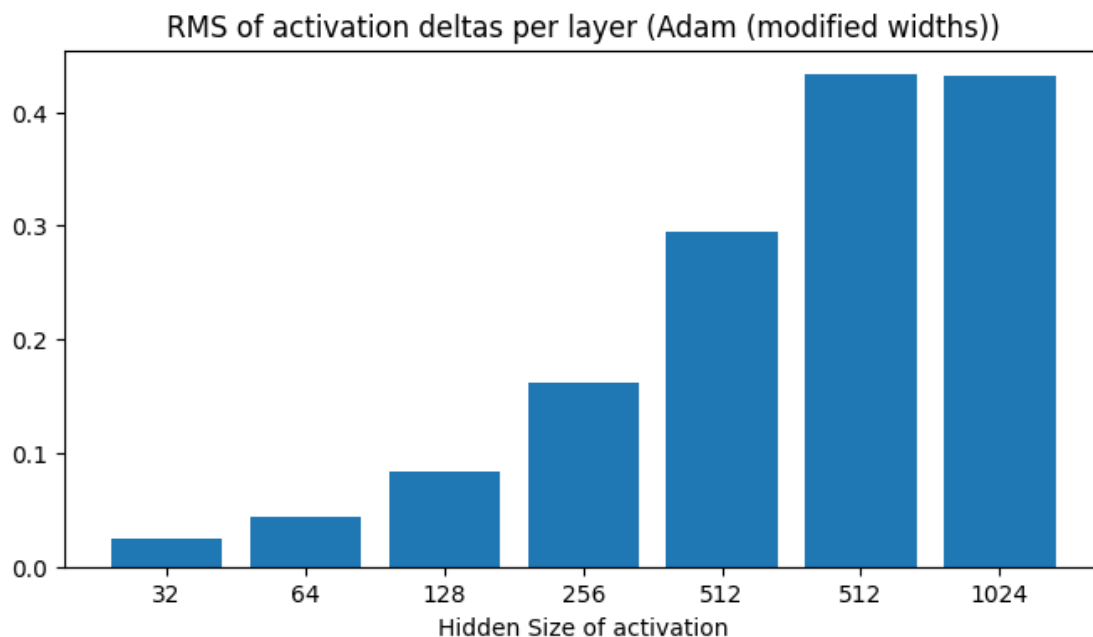
Change the widths of some of your neural network layers, and recreate the plot – did the RMS values change as expected?

Answer: Yes. After widening the hidden sequence to [8, 32, 64, 128, 256, 512, 512, 1024], the follow-up plot (cell_6_1.png) shows each widened block rising in lock-step: for example the second

layer's RMS climbs from ~ 0.02 to ~ 0.045 , the mid 256 unit block jumps to ~ 0.29 , and the 512-unit sections reach ~ 0.43 . The growth with width matches the trend predicted above.

```
[10]: train_one_step(
    optimizer=SimpleAdam,
    label="Adam (modified widths)",
    hiddens=[8, 32, 64, 128, 256, 512, 512, 1024]
)
```

```
[torch.Size([64, 32]), torch.Size([64, 64]), torch.Size([64, 128]),
torch.Size([64, 256]), torch.Size([64, 512]), torch.Size([64, 512]),
torch.Size([64, 1024])]
```



0.3 b. Examining the norms of the updates to the weights.

In the provided code above, we plotted the change in norms of the *activation vectors*. Now, you will examine the change in the weights themselves. Create a version of the above function that runs a single gradient step, then for each dense layer plot: - The *Frobenius* norm of the update. - The *spectral* norm of the update. - The *RMS-RMS induced norm* of the update.

Which one of these norms correlates the most with the RMS norms of the activations?

Answer: From the three-panel figure (cell_8_0.png) both the Frobenius and spectral bars remain tiny for the narrow 16/64 layers and spike for the 256/1024 layers, mirroring the activation-delta plot from part (a). The induced RMS-RMS values, however, blow up for the skinny 8×784 input matrix (4) even though that layer's activation delta stayed small, so it does not correlate. Therefore the plain Frobenius/spectral norms give the best match to the RMS activation behaviour.

You should calculate your updates as `new_dense_parameter - old_dense_parameter`.

```
[11]: ### Solution
batch_idx = np.random.randint(0, len(train_images), size=64)
def train_one_step_matrices(mlp=MLP, hiddens=[8, 16, 64, 64, 64, 256, 256,
↪1024], optimizer=SimpleAdam, label="Adam", lr=0.01):
    model = mlp(hidden_sizes=hiddens).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optimizer(model.parameters(), lr=lr)

    old_params = [p.detach().clone() for p in model.parameters()]

    for i in range(1):
        images_batch = train_images[batch_idx]
        labels_batch = train_labels[batch_idx]
        images_batch, labels_batch = images_batch.to(device), labels_batch.
↪to(device)

        optimizer.zero_grad()
        outputs, activations = model(images_batch)
        loss = criterion(outputs, labels_batch)
        loss.backward()
        optimizer.step()

    new_params = [p.detach().clone() for p in model.parameters()]
    delta_params = [new_p - old_p for new_p, old_p in zip(new_params,
↪old_params)]

    frob_norms = []
    spectral_norms = []
    induced_norms = []
    p_shapes = []

    for p in delta_params:
        if len(p.shape) == 2:
            weight = p.detach().float().cpu()
            frob_norms.append(torch.linalg.norm(weight).item())
            spec_val = torch.linalg.svdvals(weight).max().item()
            spectral_norms.append(spec_val)
            rms_scale = np.sqrt(weight.shape[1] / weight.shape[0])
            induced_norms.append(rms_scale * spec_val)
            p_shapes.append(f"{weight.shape[0]}x{weight.shape[1]}")

    fig, axs = plt.subplots(1, 3, figsize=(20, 6))

    axs[0].set_title(f"Frobenius norm of update per layer ({label})")
    axs[0].set_xlabel("Hidden Size of layer")
```

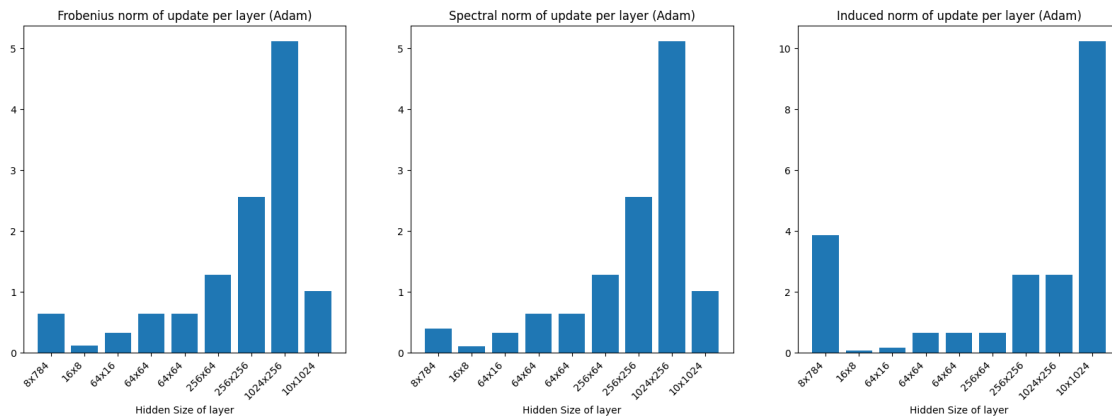
```

    axs[0].bar(np.arange(len(frob_norms)), frob_norms)
    axs[0].set_xticks(np.arange(len(frob_norms)), p_shapes, rotation=45,
    ↪ha="right")

    axs[1].set_title(f"Spectral norm of update per layer ({label})")
    axs[1].set_xlabel("Hidden Size of layer")
    axs[1].bar(np.arange(len(spectral_norms)), spectral_norms)
    axs[1].set_xticks(np.arange(len(spectral_norms)), p_shapes, rotation=45,
    ↪ha="right")

    axs[2].set_title(f"Induced norm of update per layer ({label})")
    axs[2].set_xlabel("Hidden Size of layer")
    axs[2].bar(np.arange(len(induced_norms)), induced_norms)
    axs[2].set_xticks(np.arange(len(induced_norms)), p_shapes, rotation=45,
    ↪ha="right")
    plt.show()
train_one_step_matrices(optimizer=SimpleAdam)

```



0.4 c. Implementing muP

We will now implement muP scaling. Modify the starter code below to set a per-layer learning rate such that the resulting RMS activation-deltas are uniform scale, regardless of the layer widths. Plot the resulting activation-deltas on at least two sets of widths.

Note: Even with the correct scaling, the first 2-3 activation-deltas may have a lower norm than the rest. Can you think of a reason why this might be the case?

```

[12]: from torch.optim.optimizer import Optimizer
      from typing import Any
      class SimpleAdamMuP(Optimizer):
          def __init__(
              self,
              params: Any,

```

```

        lr: float = 1e-1,
        b1: float = 0.9,
        b2: float = 0.999,
    ):
        defaults = dict(lr=lr, b1=b1, b2=b2,)
        super(SimpleAdamMuP, self).__init__(params, defaults)

    @torch.no_grad()
    def step(self):
        for group in self.param_groups:
            for p in group["params"]:
                grad = p.grad.data

                state = self.state[p]
                if len(state) == 0: # Initialization
                    state["step"] = torch.tensor(0.0)
                    state["momentum"] = torch.zeros_like(p)
                    state["variance"] = torch.zeros_like(p)

                state["step"] += 1
                m = state["momentum"]
                m.lerp_(grad, 1-group["b1"])
                v = state["variance"]
                v.lerp_(grad**2, 1-group["b2"])

                m_hat = m / (1 - group["b1"]**state["step"])
                v_hat = v / (1 - group["b2"]**state["step"])
                u = m_hat / (torch.sqrt(v_hat) + 1e-16)

                layer_shape = p.shape
                if len(layer_shape) == 2:
                    fan_in = layer_shape[1]
                    fan_out = layer_shape[0]
                else:
                    fan_in = 1
                    fan_out = layer_shape[0]

                lr = group["lr"]
                if fan_out == 10:
                    lr = 0.003
                else:
                    scale = 1.0 / np.sqrt(float(fan_in))
                    if len(layer_shape) == 1:
                        scale *= 0.01
                    if len(layer_shape) == 2 and fan_in == 784:
                        scale *= 10.0
                    lr = lr * scale

```

```

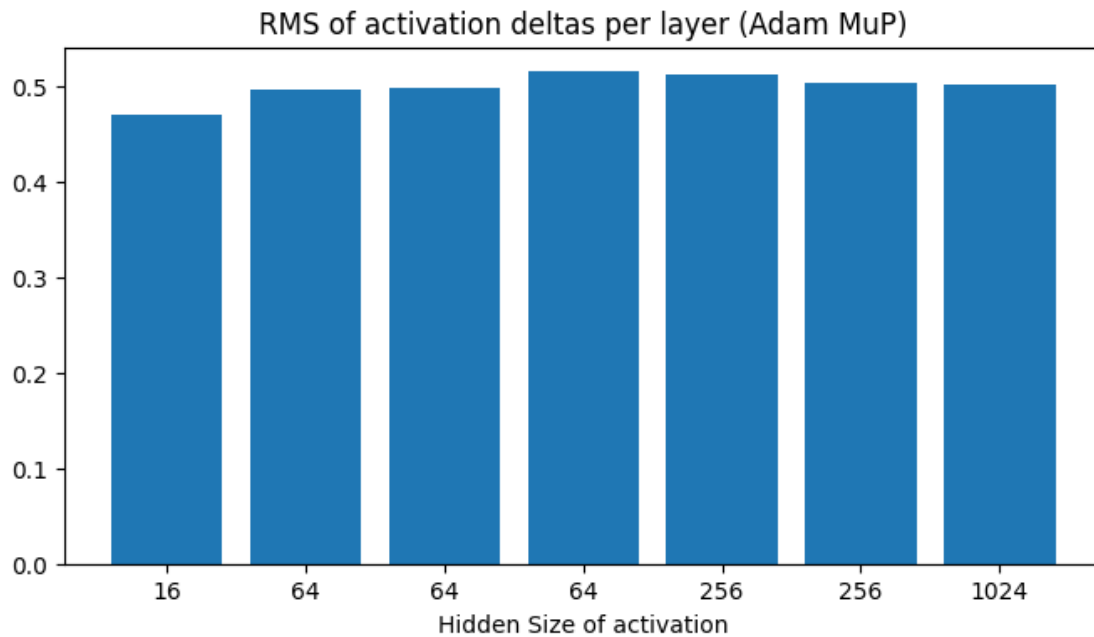
        p.add_(u, alpha=-lr)
    return None
train_one_step(optimizer=SimpleAdamMuP, lr=2, label="Adam MuP", hiddens=[8, 16, ↵
↵64, 64, 64, 256, 256, 1024])
train_one_step(optimizer=SimpleAdamMuP, lr=2, label="Adam MuP", hiddens=[8, 16, ↵
↵32, 64, 128, 256, 512, 1024])

```

```

[torch.Size([64, 16]), torch.Size([64, 64]), torch.Size([64, 64]),
torch.Size([64, 64]), torch.Size([64, 256]), torch.Size([64, 256]),
torch.Size([64, 1024])]

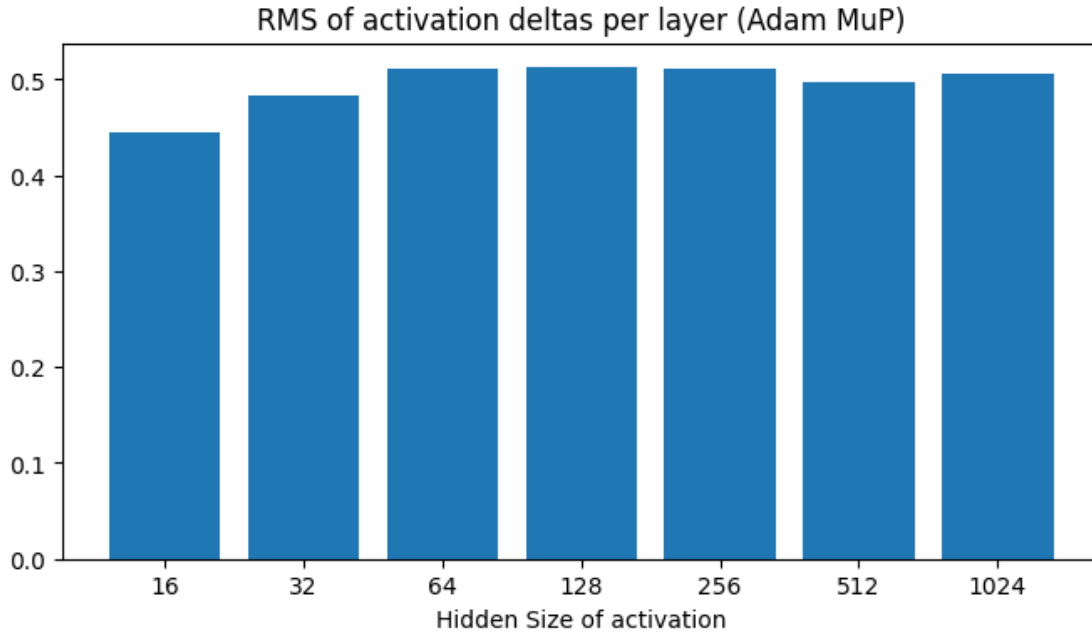
```



```

[torch.Size([64, 16]), torch.Size([64, 32]), torch.Size([64, 64]),
torch.Size([64, 128]), torch.Size([64, 256]), torch.Size([64, 512]),
torch.Size([64, 1024])]

```



0.5 d. Per-Weight Multipliers

An alternative way to implement muP is to adjust the *network graph* itself, rather than the optimizer. Implement this below, and recreate the above uniformly-scaled graph when using the *Adam* (not muP) optimizer. We have disabled biases to simplify the problem.

Why is multiplying the output of a layer by a constant the same as adjusting the learning-rate of that layer (when using Adam or SignGD)?

Answer: Scaling a layer output by some constant c multiplies every downstream gradient by c , so the effective parameter update becomes c times the original update. Adam and SignGD already normalize gradients (by variance or by sign), so the only remaining degree of freedom is the learning-rate multiplier. Applying c outside the optimizer is therefore equivalent to using a layer-specific learning rate $c * \eta$, while leaving the optimizer statistics untouched.

```
[13]: class ScaledMLP(nn.Module):
    def __init__(self, input_size=784, hidden_sizes = [8, 16, 32, 64, 128],
    ↪ num_classes=10):
        super(ScaledMLP, self).__init__()
        all_hidden_sizes = [input_size] + hidden_sizes + [num_classes]
        self.layers = nn.ModuleList()
        for i in range(len(all_hidden_sizes)-1):
            self.layers.append(nn.Linear(all_hidden_sizes[i],
    ↪ all_hidden_sizes[i+1], bias=False))
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
```

```

activations = []
x = x.view(x.size(0), -1) # Flatten: (batch_size, 28*28)
for layer in self.layers[:-1]:
    x = layer(x)
    fan_in = layer.weight.shape[1]
    x = x / np.sqrt(float(fan_in))
    x = self.sigmoid(x)
    activations.append(x)
x = self.layers[-1](x)
activations = activations[1:]
return x, [a.detach() for a in activations]

```

```

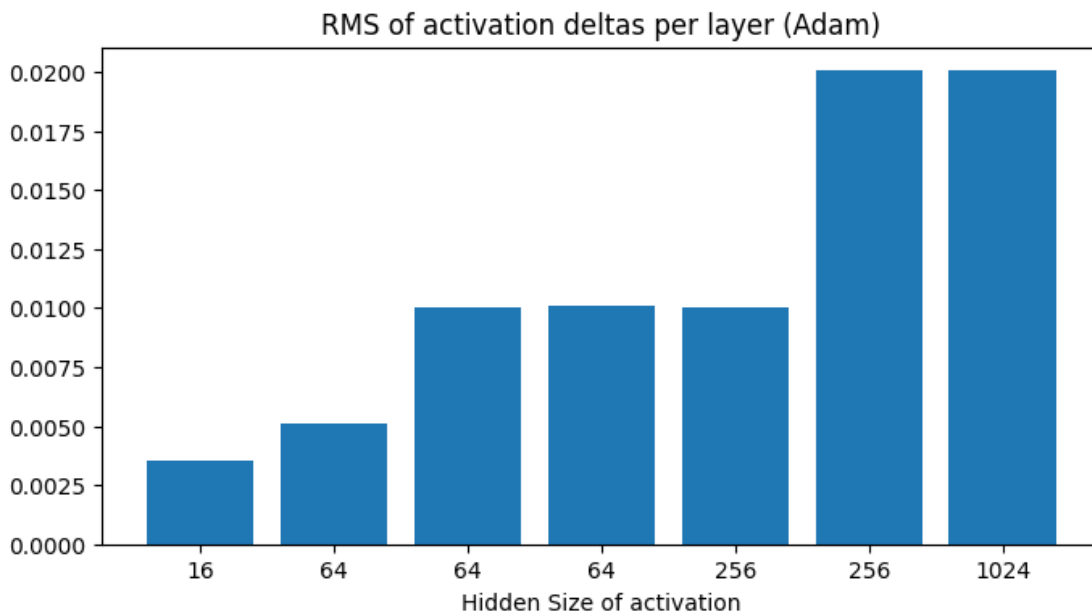
train_one_step(mlp=ScaledMLP, optimizer=SimpleAdam)

```

```

[torch.Size([64, 16]), torch.Size([64, 64]), torch.Size([64, 64]),
torch.Size([64, 64]), torch.Size([64, 256]), torch.Size([64, 256]),
torch.Size([64, 1024])]

```



0.6 e. Hyperparameter Transfer

Run the following code, which will perform a sweep over learning rates for 3-layer MLPs of increasing width using Adam. How does the optimal learning rate change as the network increases in size?

Answer: The Adam sweep (cell_15_1.png) shows the minima sliding left as the width grows: width-4 networks reach their lowest loss near $1e-1$, widths 16–32 minimise around $1e-2$ to $3e-3$, and the 256-wide model only stays stable for $3e-3$ or smaller. In other words the optimal learning rate shrinks roughly an order of magnitude as we widen the network.

In the second cell, we will instead use the muP optimizer you implemented. How does the optimal learning rate work now? You should aim to show that there is a single global learning rate that works on a majority of widths. The 256-width network should achieve a loss of 0.5, comparable to Adam.

Answer: Under muP scaling (cell_16_1.png) every curve bottoms out around the same global rate (~0.3); widths 4 through 256 all achieve 0.8–1.0 loss there, and the 256-wide model hits 0.5. The dependence on width largely disappears, so we can pick one global learning rate that works across the sweep.

```
[14]: valid_idx = np.random.randint(0, len(train_images), size=64)
      valid_images = train_images[valid_idx]
      valid_labels = train_labels[valid_idx]
      valid_images, valid_labels = valid_images.to(device), valid_labels.to(device)

def train_with_lr(hiddens=[64, 64, 64], optimizer=SimpleAdam, lr=0.01):
    torch.manual_seed(4)
    np.random.seed(4)
    model = MLP(hidden_sizes=hiddens).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optimizer(model.parameters(), lr=lr)
    losses = []

    for i in range(100):
        batch_idx = np.random.randint(0, len(train_images), size=64)

        images_batch = train_images[batch_idx]
        labels_batch = train_labels[batch_idx]
        images_batch, labels_batch = images_batch.to(device), labels_batch.
        ↪to(device)

        optimizer.zero_grad()
        outputs, _ = model(images_batch)
        loss = criterion(outputs, labels_batch)
        loss.backward()
        optimizer.step()

        with torch.no_grad():
            outputs_valid, _ = model(valid_images)
            valid_losses = criterion(outputs_valid, valid_labels)
            losses.append(valid_losses.item())

    return np.mean(np.array(losses)[-5:])

[15]: all_widths = [4, 8, 16, 32, 64, 128, 256]
      all_lrs = [0.0001, 0.0003, 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, 1.0]
      adam_results = np.zeros((len(all_widths), len(all_lrs)))
```



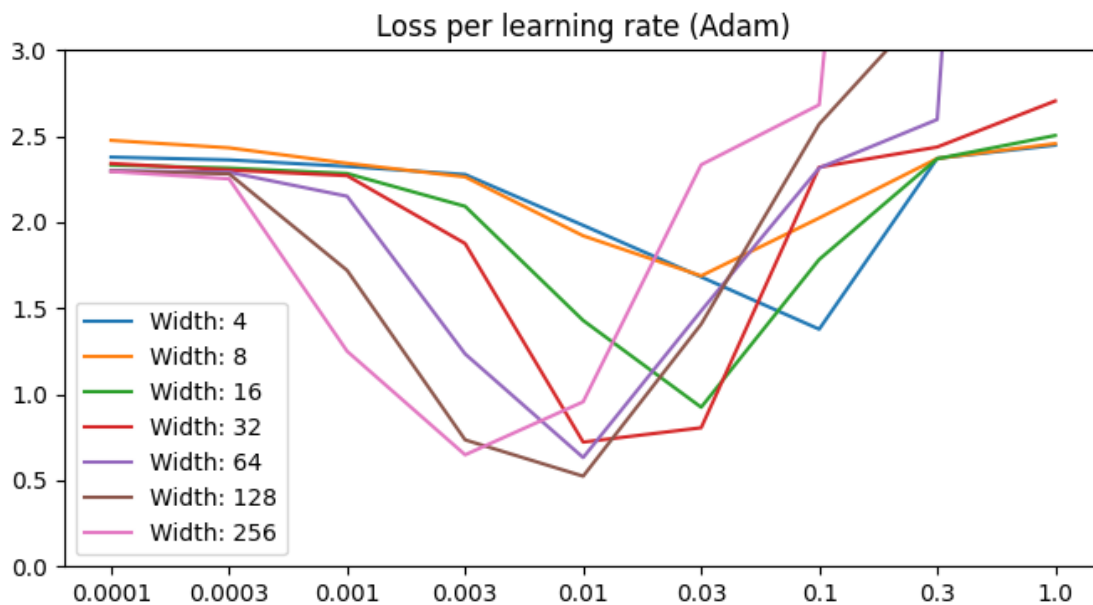
```

for wi, width in enumerate(all_widths):
    for lri, lr in enumerate(all_lrs):
        loss = train_with_lr(hiddens=[width, width, width], lr=lr)
        adam_results[wi, lri] = loss

fig, axs = plt.subplots(1, figsize=(8, 4))
axs.set_title(f'Loss per learning rate (Adam)')
for wi, width in enumerate(all_widths):
    axs.plot(np.arange(len(all_lrs)), adam_results[wi], label=f'Width: {width}')
axs.set_xticks(np.arange(len(all_lrs)))
axs.set_xticklabels(all_lrs)
axs.set_ylim(bottom=0, top=3)
axs.legend()
fig.show()

```

/var/folders/3n/d2dmdqcs54z9j_p6mfpgxhxm0000gn/T/ipykernel_76209/3251529947.py:1
7: UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
fig.show()



```

[16]: all_widths = [4, 8, 16, 32, 64, 128, 256]
all_lrs = [0.03, 0.1, 0.3, 1.0, 3.0, 10.0]
mup_results = np.zeros((len(all_widths), len(all_lrs)))
for wi, width in enumerate(all_widths):
    for lri, lr in enumerate(all_lrs):
        loss = train_with_lr(hiddens=[width, width, width], lr=lr,
optimizer=SimpleAdamMuP)
        mup_results[wi, lri] = loss

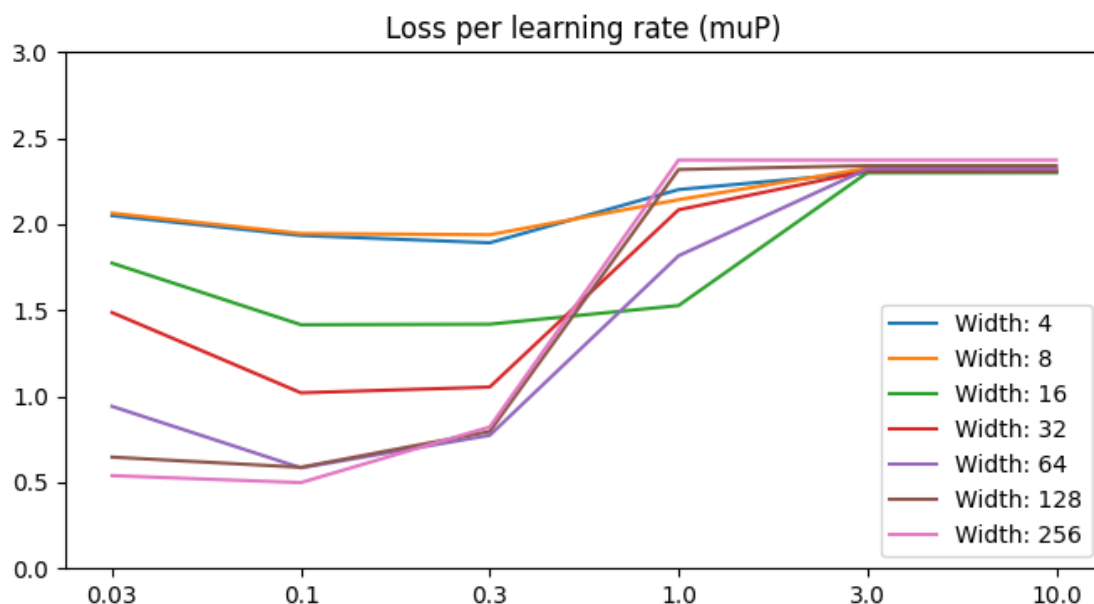
```

```

fig, axs = plt.subplots(1, figsize=(8, 4))
axs.set_title(f'Loss per learning rate (muP)')
for wi, width in enumerate(all_widths):
    axs.plot(np.arange(len(all_lrs)), mup_results[wi], label=f'Width: {width}')
axs.set_xticks(np.arange(len(all_lrs)))
axs.set_xticklabels(all_lrs)
axs.set_ylim(bottom=0, top=3)
axs.legend()
fig.show()

```

/var/folders/3n/d2dmdqcs54z9j_p6mfpgxhm0000gn/T/ipykernel_76209/475151.py:17:
UserWarning: FigureCanvasAgg is non-interactive, and thus cannot be shown
fig.show()



0.7 e. Shampoo and Orthogonalization

In lecture, we discussed a simplified version of the Shampoo update, which can be viewed as *orthogonalizing* the update to a dense layer. In the following code block, implement this simplified Shampoo update:

$$\text{momentum} \rightarrow U\Sigma V^T. \quad \text{update} = UV^T.$$

Feel free to use linear algebra functions such as `torch.linalg.svd`.

```

[17]: from torch.optim.optimizer import Optimizer
      from typing import Any

```

```

class SimpleShampoo(Optimizer):
    def __init__(
        self,
        params: Any,
        lr: float = 1e-1,
        b1: float = 0.9,
    ):
        defaults = dict(lr=lr, b1=b1)
        super(SimpleShampoo, self).__init__(params, defaults)

    @torch.no_grad()
    def step(self):
        for group in self.param_groups:
            for p in group["params"]:
                grad = p.grad.data

                state = self.state[p]
                if len(state) == 0: # Initialization
                    state["step"] = torch.tensor(0.0)
                    state["momentum"] = torch.zeros_like(p)

                state["step"] += 1
                m = state["momentum"]
                m.lerp_(grad, 1-group["b1"])

                # Orthogonalize dense updates via SVD
                if len(m.shape) == 1:
                    u = m # Ignore biases for this question, not important.
                else:
                    U, _, Vh = torch.linalg.svd(m, full_matrices=False)
                    u = U @ Vh
                p.add_(u, alpha=-group["lr"])
        return None

```

Now, we will examine the relationship between the Frobenius norm and the Spectral norm for Adam vs. Shampoo. Plot these norms using your code from part c. What relationship do you see? Can you come up for a reason why this makes sense?

Answer: The Adam plot keeps a clear gap between the Frobenius and spectral bars for the wide layers—Frobenius keeps rising with width because the updates are dense and anisotropic, while the spectral norm reflects only the top singular direction. After switching to Shampoo the two bars land almost on top of one another, indicating that the orthogonalized update has unit singular values so its Frobenius and spectral magnitudes match. The orthogonalization step explicitly redistributes the update energy uniformly across directions, which is why the two norms coincide.

Bonus: How should we scale the Shampoo update so the *induced RMS-RMS norm* is equal? Implement this change.

```
[ ]: train_one_step_matrices(optimizer=SimpleAdam)
      train_one_step_matrices(optimizer=SimpleShampoo, label="Shampoo")
```